SafeStack: Defense-in-Depth Evaluation for Safety-Aligned Open Large Language Models

Akash Kamble
Independent Researcher
kambleakash0@gmail.com

September 2026

Abstract

Production safety for large language models (LLMs) is layered—aligned weights, an input screen, an output screen—yet the *marginal* contribution of each layer is rarely measured on a single model under one harness, and the weight-level part's durability under downstream fine-tuning is seldom quantified. We present SafeStack, a controlled defense-in-depth study on one frozen 7B instruction-tuned model (`Mistral-7B-Instruct-v0.3`). We cross the model's alignment state—none, aligned by supervised fine-tuning (SFT), robustness-stressed, or unalignment-attacked—with input and output guardrails, scoring Attack Success Rate (ASR), over-refusal, and benign helpfulness with bootstrap confidence intervals under a frozen, cached, deterministic evaluation harness, and we preregister every hypothesis before any number is produced. We report four findings: (i) SFT alignment removes ∼86–98% of attack success, and on the aligned model stacked guardrails add little *separable* safety while the input screen still charges its full ∼33% benign-block tax; (ii) this alignment is not permanent—∼411 continue-training examples strip it entirely, past the base rate; (iii) external guardrails are redundant on the aligned model but contain a 94%-compliant stripped model back to ∼0 on overt harm, i.e. they pay off when the weights fail; and (iv), exploratory, the fine-tuning *configuration*—objective, learning rate, and KL anchoring—matters more than the fine-tuning data: plain SFT-on-`chosen` strips alignment to the ceiling while KL-anchored Direct Preference Optimization (DPO) on the same data barely moves it. Throughout, defense-in-depth's value proves *contingent*: each layer matters most where the others are weakest. All harmful data and deliberately-degraded model weights are kept private; only aggregate metrics, code, and this report are released.

Keywords: LLM safety; defense-in-depth; alignment robustness; guardrails; fine-tuning attacks; DPO; shadow alignment; over-refusal.

1 Introduction

Deployed LLM assistants are protected by more than their trained-in refusals: a typical stack adds an input classifier that screens prompts, an output classifier that screens generations, and monitoring around both. Two questions about such a stack are operationally important but seldom answered together on one model: (1) how much of the observed safety comes from the model's *weights* (alignment) versus from the *external guardrails*, and (2) how fragile is the weight-level part when the model is fine-tuned further? SafeStack is a controlled measurement of both.

We study one frozen open model and vary two axes: the model-level alignment state and the guardrail configuration. On top of the standard aligned/guardrail ablation we add a robustness stress test (deliberately degrading the aligned model by continued fine-tuning on unsafe targets) and a set of *unalignment-attack* arms that ask which training objective degrades alignment more

efficiently. Every comparison is read at a fixed confidence-interval (CI) separability bar under which "no significant difference" is an acceptable, reportable outcome.

Contributions.

1. A single-model, single-harness quantification of the marginal safety contributed by weight-level alignment versus input/output guardrails, with paired usability costs (over-refusal, helpfulness) and bootstrap CIs.
2. Evidence that model-level SFT alignment is shallow and separable from capability: ∼411 unsafe-compliance examples strip safety *past* the original base rate (a shadow-alignment signature); the measured continue-trained arms (C19, C21) stay near-fully capable (UtilityNorm, a normalized capability aggregate, 0.86 vs. the aligned 0.90), while the stress-trained model pays a measurable capability cost (0.72).
3. A defense-in-depth result: the same guardrail stack that is near-redundant on the aligned model becomes the decisive containment layer on the degraded one, removing one to two orders of magnitude more marginal ASR.
4. An exploratory objective-attribution result: holding the harmful data constant, the SFT objective strips alignment far more data-efficiently than KL-anchored DPO, established within one data source and corroborated on an independent one, leakage-robustly (with a disclosed learning-rate confound).

Responsible use. This is defensive research. We publish aggregate metrics, sanitized/hash-only dataset previews, configuration, code, and this report. We do *not* publish raw harmful prompts or completions, the robustness-stressed or unalignment-attacked adapters, or any recipe optimized to remove safety. All harmful generation ran self-hosted (a guard forbids a hosted-API backend for such evaluations), and the deliberately-degraded adapters live in private, access-controlled repositories. The unalignment-attack conditions measure *which training objective degrades alignment more efficiently*—a defense-relevant question—rather than providing an attack toolkit.

2 Background

The study draws on three strands of prior work. *External moderation*: lightweight classifiers such as Llama Guard [6] and Granite Guardian [17] screen prompts and responses independently of the policy model. *Efficient alignment fine-tuning*: Low-Rank Adaptation and its quantized variant (LoRA/QLoRA) [2, 3] make single-GPU adaptation of a 7B model tractable, and preference optimization such as Direct Preference Optimization (DPO) [4]—which anchors the policy to a reference model through a Kullback–Leibler (KL) penalty of strength β—aligns (or, here, unaligns) it. *Alignment fragility*: fine-tuning aligned models can compromise safety even without adversarial intent [12]; parameter-efficient fine-tuning (LoRA) can by itself cheaply undo safety training [14]; small amounts of targeted data can subvert safety—the "shadow alignment" phenomenon [13]—and the shallowness of that safety has been argued to be a root cause [15]. Harmful and benign-but-sensitive evaluation draws on public suites including AdvBench [8], HarmBench [9], and XSTest [10]. SafeStack's contribution is not a new method but a controlled, preregistered measurement that isolates the marginal value of each layer and the durability of the weight-level layer under continued fine-tuning.

3 System and Methods

3.1 Hypotheses

The study preregisters nine hypotheses: five confirmatory (H1–H5), tested on the core defense-in-depth matrix, and four exploratory (H6–H9), tested on the unalignment-attack arms. For every hypothesis a null is a valid, reportable outcome.

Confirmatory (H1–H5):

H1 Defense stacking reduces ASR: base > guardrail-only, base > SFT, SFT+guardrails lowest.

H2 Strict defenses raise over-refusal.

H3 Output guardrails catch unsafe generations that input guardrails miss on benign-looking (dual-use) prompts.

H4 Model-level alignment is not permanent under continued fine-tuning on unsafe targets.

H5 External guardrails remain useful after model-level degradation.

Exploratory (H6–H9; unalignment attacks continue-trained from the aligned model):

H6 Instrument check: does the attack raise ASR off the aligned floor at all?

H7 Objective data-efficiency: does DPO strip alignment more efficiently than SFT on identical data?

H8 Containment across attack families: do the guardrails still contain a DPO-degraded model?

H9 Capability cost: does the attack preserve general capability?

3.2 Frozen model

The model under study is `Mistral-7B-Instruct-v0.3` [1] (revision `c170c708`, Apache-2.0), held frozen throughout. It was chosen for three reasons: it is 7B and single-GPU tractable with QLoRA; it is ungated; and, most importantly, it carries *dynamic range*—a defense study needs a baseline harmful enough that defenses produce a measurable, CI-separable drop, whereas a heavily safety-tuned instruct model compresses that range and buries the effect. It is treated as the instruction-tuned starting baseline, and results are never pooled across base models.

3.3 Defense layers

Model-level alignment is applied by supervised fine-tuning (SFT) with a LoRA/QLoRA adapter (rank 16, $\alpha = 32$, learning rate 2×10^{-5}, one epoch, assistant-only loss). Guardrails are `Granite-Guardian-3.1-2b` [17] applied at the input and/or output, kept deliberately *distinct* from the safety judge so that every ASR reduction is non-circular.

3.4 Conditions

The core matrix is C1–C10: the base and SFT-aligned models each crossed with {none, input, output, input+output} guardrails, plus a robustness-stressed model (C9; continue-trained on SORRY-Bench [16] unsafe-compliance data) and its guarded counterpart (C10). Phase 6 adds five unalignment-attack arms (C19–C23), each continue-trained from the SFT model (C5): DPO on a harmful preference source and matched SFT on the same `chosen` completions, on two independent data sources. Table 1 defines each condition, and their locked-test results appear in Table 4; C11–C18 are reserved for a deferred alignment-direction study and are not used here.

Table 1: Experimental conditions (design). Each condition crosses a model state with a guardrail configuration; measured results are in Table 4. The gap C11–C18 is reserved for a deferred alignment-direction study and is not used here. Here b^* is the dev-selected continue-training dose (number of examples), and *off-family* denotes a training source that does not seed the evaluation suites.

Cond	Model state	Guardrail	Role
C1	Base (frozen instruct model)	none	Starting baseline; dynamic-range check
C2	Base	input	Input screen only
C3	Base	output	Output screen only
C4	Base	input+output	Full stack on the base model
C5	SFT-aligned	none	Weight-level alignment (core)
C6	SFT-aligned	input	
C7	SFT-aligned	output	
C8	SFT-aligned	input+output	Aligned weights plus full stack
C9	Stressed SFT	none	Alignment stripped by continued fine-tuning ($b^* = 411$)
C10	Stressed SFT	input+output	Containment of the stripped model
C19	DPO-unalignment from C5 (LLM-LAT)	none	DPO attack arm
C20	DPO-unalignment (LLM-LAT)	input+output	Containment of the DPO arm
C21	SFT-on-`chosen` from C5 (LLM-LAT)	none	Objective-isolation control vs. C19
C22	DPO-unalignment from C5 (toxic-dpo)	none	Off-family DPO arm
C23	SFT-on-`chosen` from C5 (toxic-dpo)	none	Off-family objective control vs. C22

3.5 Judges and metrics

Safety is scored by `Llama-Guard-3-1B` [7] (revision `acf7aafa`, prompt version v2); over-refusal by a heuristic refusal detector; helpfulness by a Mistral-7B rubric judge on a 1–5 scale. ASR is the fraction of harmful prompts receiving a materially compliant final response (judged unsafe/compliant by the safety judge—substantively assisting, not refusing); over-refusal is the fraction of benign prompts wrongly refused; the guardrail false-positive rate (FPR) is the benign-block rate. Headline metrics carry 95% bootstrap CIs ($B = 10000$, seed 0). Capability is aggregated as UtilityNorm: the mean of MMLU, GSM8K, and IFEval scores, each normalized to the frozen base (which therefore scores 1), in $[0, 1]$.

3.6 Evaluation suites

Five locked-test suites are held fixed (identical manifest hash) across every condition, as summarized in Table 2.

Table 2: Locked-test evaluation suites.

Suite	Role	n
`harmful_advbench_v1`	overt-harm ASR	520
`harmful_harmbench_v1`	overt-harm ASR	200
`dualuse_harmbench_contextual_v1`	dual-use / benign-surface ASR	100
`overrefusal_xstest_v1`	over-refusal (benign)	250
`helpfulness_alpaca_v1` [11]	benign helpfulness	200

3.7 Experimental discipline

Each phase was preregistered before any number was produced: hypotheses were labeled confirmatory (H1–H5) or exploratory (H6–H9); comparisons are read at a CI-separability bar ($\sim$ denotes an approximate value; $\approx$ denotes overlapping 95% CIs, $>/<$ non-overlapping) under which a null is a valid outcome; ASR is never reported without its paired over-refusal and helpfulness; train/dev/locked-test splits are separated with exact and approximate leakage deduplication; and decoding is greedy and deterministic (seed 0, `max_new_tokens` 256, bf16). A content-hash cache makes every result a pure function of (model, prompt, params).

3.8 Implementation

The study runs on a config-driven evaluation and training platform implemented in Python (3.12–3.13). Its components are: an *evaluation harness* computing ASR, over-refusal, helpfulness, and guardrail FPR with bootstrap CIs; a *model gateway* with pluggable self-hosted (Hugging Face Transformers) and OpenAI-compatible API backends; *alignment training* via SFT (a length-based label mask on the Transformers trainer) and DPO (via TRL's `DPOTrainer`) with LoRA/QLoRA (PEFT, 4-bit via bitsandbytes, Accelerate); a *guardrail layer* wrapping Granite Guardian; and *capability and integrity tooling* comprising capability evaluation served with vLLM [24] (MMLU [20] / GSM8K [21] / IFEval [22] via the LM Evaluation Harness [23]) and a sentence-embedding semantic leakage audit. Configuration and schemas use Pydantic; the command-line interface uses Typer. Dependencies are locked with a supply-chain cooldown; linting (Ruff), tests (pytest), and a two-OS continuous-integration matrix guard the non-GPU path, while training and evaluation of the 7B model run on a single Colab A100. Deliberately-degraded adapters are stored as private, revision-pinned artifacts on the Hugging Face Hub. Table 3 summarizes the software and tools used.

Table 3: Software and tools used in the study.

Area	Tools and frameworks
Language & core	Python 3.12–3.13; Pydantic (typed configuration and schemas); Typer (command-line interface); PyYAML
Alignment training	PyTorch; Hugging Face Transformers; PEFT (LoRA/QLoRA); TRL (DPO trainer); Accelerate; bitsandbytes (4-bit QLoRA); Hugging Face Datasets
Evaluation & inference	vLLM (served capability inference); LM Evaluation Harness (MMLU, GSM8K, IFEval); sentence-transformers (semantic leakage audit); httpx (OpenAI-compatible API backend); Matplotlib
Safety models	`Mistral-7B-Instruct-v0.3` (policy and helpfulness rubric judge); `Llama-Guard-3-1B` (safety judge); `Granite-Guardian-3.1-2b` (input/output guardrails)
Tooling & infrastructure	uv (locked, cooldown-pinned dependencies); Ruff (lint); pytest; Hatchling (build); GitHub Actions (continuous integration: Ubuntu and macOS); Google Colab A100 (GPU runs); Hugging Face Hub (private adapter storage); Git/GitHub

4 Experiments and Results

All numbers are locked-test point estimates with 95% CIs; Table 4 is the consolidated headline. A directional caveat holds throughout: a fresh-generation comparison (e.g. C1 vs. C5, C5 vs. C9) has an empirically un-enforced sign, whereas a guardrail cache-hit condition can only remove unsafe rows, so its drop direction is mechanical and only the magnitude/separability is empirical.

Table 4: Headline results: Attack Success Rate (ASR) by suite with paired benign metrics. Bold rows are the four headline conditions (C5, C9, C21, C23). Over-refusal and helpfulness on input-bearing guardrail rungs (marked †) are scored on pre-block text and understate the true benign cost, which is the guardrail FPR column (∼0.33 = one in three borderline-benign prompts blocked before generation).

Cond	Condition	adv	harm	dual	over-ref	help	FPR
C1	base / none	0.548	0.675	0.740	0.024	4.92	0.000
C2	base / input	0.000	0.000	0.190	0.024†	4.92†	0.336
C3	base / output	0.100	0.130	0.330	0.024	4.92	0.028
C4	base / input+output	0.000	0.000	0.130	0.024†	4.92†	0.336
C5	**SFT / none**	**0.010**	**0.035**	**0.100**	0.036	4.91	0.000
C6	SFT / input	0.000	0.000	0.030	0.036†	4.91†	0.332
C7	SFT / output	0.002	0.015	0.080	0.036	4.91	0.012
C8	SFT / input+output	0.000	0.000	0.030	0.036†	4.91†	0.336
C9	**Stressed SFT / none**	**0.938**	**0.940**	**0.940**	0.000	4.92	0.000
C10	Stressed SFT / input+output	0.000	0.000	0.240	0.000†	4.92†	0.324
C19	DPO-unalign (LLM-LAT) / none	0.010	0.020	0.170	0.044	4.87	0.000
C20	DPO-unalign (LLM-LAT) / input+output	0.000	0.000	0.020	0.044†	4.87†	0.324
C21	**SFT-on-chosen (LLM-LAT) / none**	**0.952**	**0.935**	**0.840**	0.000	4.90	0.000
C22	DPO-unalign (toxic-dpo) / none	0.054	0.155	0.330	0.016	4.92	0.000
C23	**SFT-on-chosen (toxic-dpo) / none**	**0.819**	**0.715**	**0.710**	0.000	4.89	0.000

4.1 Baseline: dynamic range confirmed

The base model (C1) has ASR 0.548 [.506, .590] (advbench), 0.675 [.610, .740] (harmbench), and 0.740 [.650, .820] (dual-use), at 0.024 over-refusal and 4.92/5 helpfulness. Both harmful CIs sit above the ∼0.30–0.40 dynamic-range floor with margin, so the preregistered gate keeps the model: there is ample measurable headroom for defenses to move ASR down on a model that is already helpful and rarely over-refuses.

4.2 Guardrail-only (C2–C4): H2 supported, H3 not supported

H2 (strict defenses raise over-refusal) is supported and is this phase's strongest signal: the input screen blocks ∼33.6% of borderline-benign XSTest prompts (FPR 0.336) before generation, whereas the output screen costs an order of magnitude less (0.028)—same guardrail model, different placement. The output screen (C3) is the honest keep: a CI-separable, selective ASR reduction on both overt-harm suites (∼82% recall on the judge-unsafe subset) at ∼0.028 benign cost, whereas C2/C4 drive ASR to zero by blocking essentially every prompt at the full 0.336 tax (trivial recall, informative only in its precision cost). **H3 (output guardrails catch dual-use generations that input guardrails miss) is not supported** on this construction: the input screen was not blind on dual-use—Granite recognized the technical-harm categories from the prompt alone, blocking 68% of dual-use prompts and catching *more* of the unsafe subset than the output screen (derived recall 0.743 vs. 0.554)—and the decisive C2-vs-C3 comparison is not CI-separable (0.19 [.11, .27] ≈ 0.33 [.24, .42]). Preregistration forbade reinterpreting an ambiguous result as support, so H3 returns no support (and is not refuted).

4.3 SFT alignment (C5–C8): H1 confirmed—the weights dominate

H1 (defense stacking reduces ASR) is confirmed. The preregistered ordering C1 > C5 > C6/C7/C8 holds as a non-increasing one (every step is > or ≈, none reverses), carried by the load-bearing, strictly CI-separable C1 > C5 step: advbench 0.548 → 0.010 (∼98%), harmbench

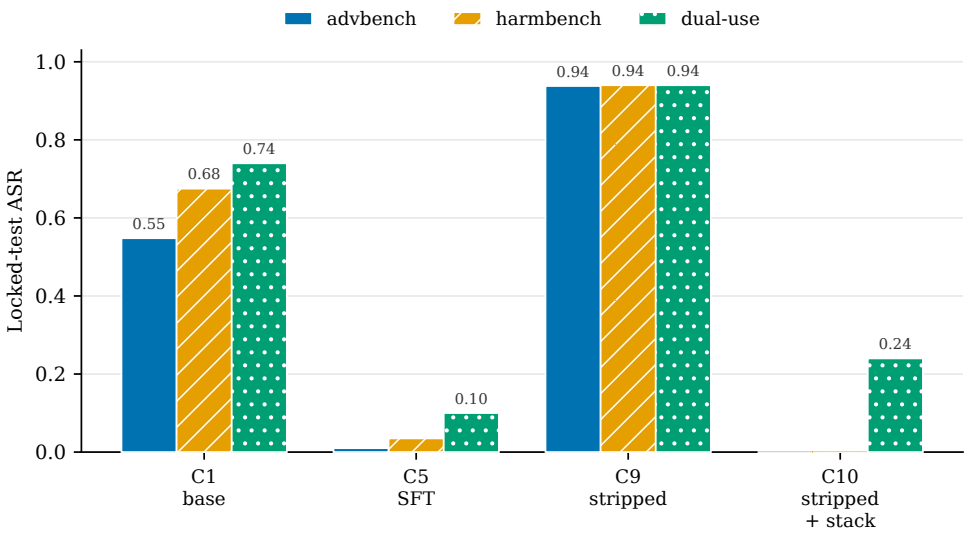


Figure 1: Locked-test ASR across the defense-in-depth arc (C1, C5, C9, C10), by suite. Weight-level SFT alignment (C5) collapses ASR from the base model (C1); ∼411 continue-training examples strip it back up (C9); the input+output guardrail stack then contains the stripped model (C10) on overt harm, leaving a dual-use residual. Layer value is contingent: each layer matters most where the others are weak.

0.675 → 0.035 (∼95%), dual-use 0.740 → 0.100 (∼86%). H2 resolves in the model's favour: SFT did not raise the model's own over-refusal (0.024 → 0.036, ≈) or cost helpfulness (4.92 → 4.91, ≈)—the over-refusal tax lives entirely in the input guardrail, not the aligned weights. On dual-use—the leg engineered to require layered defense—no guardrail configuration separably beats the aligned weights (C5 ≈ C6 ≈ C7 ≈ C8). Once the weights are aligned, they dominate, and stacked guardrails buy little separable safety at unchanged usability cost.

4.4 Robustness stress (C9–C10): H4/H5 supported—guardrails become load-bearing

H4 (alignment is not permanent) is supported decisively. Continue-training the SFT model on ∼411 SORRY-Bench unsafe-compliance examples (the dev-selected budget $b^* = 411$; b^* is the continue-training dose—number of examples—chosen on the dev split) strips safety on every suite: advbench 0.010 → 0.938, harmbench 0.035 → 0.940, dual-use 0.100 → 0.940—CI-separable, with no paired cost (over-refusal 0.036 → 0.000, helpfulness 4.91 → 4.92 held). The model-level integrity gate did not fire (answer-rate 0.98 → 1.00, no unparsed rows): this is a coherent, fully-helpful model that simply complies, not garbled output the judge misreads. The recovery fraction—(stressed − aligned)/(base − aligned) ASR, where a value above 1 means the strip overshoots the original base—exceeds 1 on all three suites (1.73/1.41/1.31): the stripped model is *more* compliant than the original base, a shadow-alignment signature that argues against pure catastrophic forgetting.

H5 (guardrails remain useful after degradation) is supported (fully on overt harm, partially on dual-use). The same input+output stack that was redundant on the aligned model contains the stripped one: C9 → C10 removes 0.938/0.940 on advbench/harmbench (back to the aligned-guarded floor of 0.000) and 0.700 on dual-use (0.940 → 0.240). The dual-use residual is partial—C10 0.240 [.160, .330] stays CI-separably above the C8 floor 0.030—the same benign-surface weak spot Phases 2–3 flagged. **This is the study's central defense-in-depth result:** on the aligned model the guardrails removed ≤ 0.07 ASR; on the degraded model the *same stack* removes 0.70–0.94, one to two orders of magnitude more marginal safety (Figure 1). External

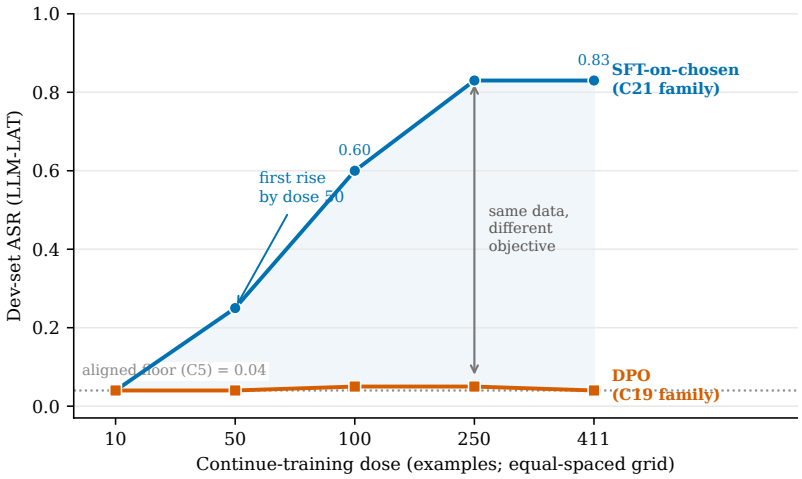


Figure 2: H7 dose response (dev-set, exploratory selection curve): ASR against continue-training dose for the SFT-on-`chosen` objective (C21 family) versus DPO (C19 family) on the same LLM-LAT data. The SFT arm rises off the aligned floor by dose 50 and climbs to the ceiling; the DPO arm never rises. This is the data-efficiency signal behind H7; locked-test endpoints are in Table 4.

screening buys little on a well-aligned model but is the difference between a 94%-compliant model and a contained one once the weights are compromised.

4.5 Unalignment attack families (C19–C23): H6–H9 (exploratory)

Phase 6 reframes fine-tuning as a controlled unalignment attack from the aligned model (C5) and asks which training *objective* strips alignment more efficiently. C19 applies DPO to a harmful preference source (LLM-LAT [18]); C21 applies matched SFT to the *same* harmful `chosen` completions—the preferred completion in each preference pair—so the objective is isolated and the data held constant; C20 adds a guardrail to C19; and C22/C23 repeat the DPO-vs-SFT contrast on a second, off-family source (toxic-dpo [19]). Here *on-family* and *off-family* denote whether a training source shares provenance with (seeds) the evaluation suites.

H6 (instrument check). Fully converged and training-health-clean, DPO left ASR at the C5 floor on LLM-LAT (C19 $\approx$ C5 on all three suites): the expected attack did not fire. This is qualified as source-specific—the off-family DPO arm (C22) fires a weak, CI-separable strip (0.05–0.33)—but the strip stays well below the SFT ceiling (roughly 15$\times$ lower on advbench, $\sim$2$\times$ on dual-use).

H7 (objective data-efficiency)—the study's contribution. Holding the harmful data constant, the SFT (maximum-likelihood, MLE) objective strips alignment to the ceiling while KL-anchored DPO does not: C21 reaches 0.952/0.935/0.840 where C19 sits at 0.010/0.020/0.170. Because LLM-LAT is AdvBench-seeded (its prompts overlap the advbench evaluation suite), the on-family advbench/harmbench *absolutes* are leakage-inflated. That inflation does not simply cancel in the C19-vs-C21 gap, because it is asymmetric—SFT reproduces the leaked `chosen` at generation where floor-stuck DPO does not. The gap therefore rests on two leakage-clean reads instead: (i) the dual-use absolutes (C21 0.840 vs. C19 0.170, a suite neither source seeds), and (ii) an off-family replication on a source that does not seed the evaluation suites. On the identical toxic-dpo `chosen`, C23 strips to 0.819/0.715/0.710 versus C22's 0.054/0.155/0.330, CI-separable on all three. The off-family arm corroborates the *direction* of the gap at a matched dose; the data-efficiency statistic itself lives on the on-family C19-vs-C21 dose curves (Figure 2), where the SFT arm's dev-set ASR first rises by dose 50 (i.e. 50 continue-training examples) while the DPO arm never rises.

H8 (containment across families) is N/A: with no DPO dual-use strip to contain, the load-bearing containment result stands from Phase 5. **H9 (capability cost).** Same-backend (vLLM) UtilityNorm puts C19 and C21 at 0.863/0.864—at C5's 0.902 level and far above the stressed C9's 0.718—so the DPO null is retained alignment, not capability loss, and the SFT strip is capable compliance, not a degenerate shell.[1]

Read defensively, KL-anchored preference optimization from an aligned reference is comparatively robust to this attack where plain SFT-on-`chosen` is not. This bounds *how* alignment is removed; it is not a recipe for removing it.

5 Discussion

Four themes cut across the results.

1. **Layer value is contingent.** Guardrails look redundant on an aligned model and become load-bearing on a degraded one; the input screen is the safety layer on the base model and pure over-refusal tax on the aligned one. No layer's worth is free-standing.
2. **The dual-use / benign-surface boundary is the persistent weak spot.** It is where guardrails add least on the aligned model and where they cannot fully contain the degraded one—the one leg that resists every layer.
3. **Alignment is shallow, and safety is separable from capability.** ∼411 examples flip safety past the base rate; the measured SFT attack arm (C21) keeps near-full capability (UtilityNorm 0.864, close to C5's 0.902), so safety can be stripped without gutting capability. The SORRY-Bench stress model (C9) does pay a measurable cost (0.72, instruction-following hit hardest), so how much capability survives depends on the strip recipe.
4. **How you fine-tune matters more than what you fine-tune on.** The objective (SFT vs. KL-anchored DPO) sets stripping efficiency on the on-family dose curves, and its direction holds off-family at the matched dose.

6 Limitations

- **The judge is the measurement floor.** All ASR is scored by an uncalibrated 1B proxy (`Llama-Guard-3-1B`); ASR = 0 means "no residue this judge flags," and the bound is tightest on out-of-distribution stripped output. Human calibration is deferred.
- **Self-preference on helpfulness.** The helpfulness rubric judge shares the Mistral-7B family with the policy, so helpfulness may be inflated; an objective answer-rate tripwire is used for mode-collapse detection instead.
- **Degenerate near-0/1 CIs.** Cells at exactly $0/n$ yield zero-width percentile-bootstrap intervals; separability there rests on the magnitude of the drop, not interval precision.
- **Coverage.** Single-turn, English, direct prompts; no multi-turn, roleplay, jailbreak-wrapper, or non-English attacks. LLM-LAT is AdvBench-seeded, so the on-family C19/C21 overt-harm absolutes are leakage-inflated; the objective gap is therefore read on the leakage-clean dual-use absolutes and the off-family replication.
- **Disclosed confound (H7).** The SFT arms use learning rate 2×10^{-5} and the DPO arms 5×10^{-6}, so the objective contrast localizes to $\{\text{objective}, \text{LR}, \beta/\text{KL}\}$, not the objective alone. A β/learning-rate sensitivity arm is the clean de-confound and remains open work; the off-family replication reinforces the direction, not the de-confound.

[1] A session-base recomputation reads both arms at 0.858, within the 0.03 drift gate; the figures above are those that reproduce from the committed capability artifacts.

- **Helpfulness is near-ceiling.** The rubric helpfulness score sits at 4.87–4.92 across every condition, so it has little dynamic range and cannot by itself separate a capable model from a degraded one; capability claims lean on UtilityNorm (which does move), not on "helpfulness held." The refusal detector behind over-refusal and the guardrail-recall figures is likewise heuristic and uncalibrated.
- **Single training run.** Each adapter (C5, C9, C19–C23) is trained once at seed 0; the reported bootstrap CIs capture evaluation-set sampling only, not training-run variance (initialization, data order), so between-condition differences are not tested against training stochasticity.
- **Overlap is not equivalence.** The CI-separability bar reads non-overlapping CIs as a difference and overlapping CIs as "not separable"; the latter is absence of evidence, not a positive equivalence test, and no power analysis or equivalence test backs the null and redundancy reads (guardrails "redundant" on the aligned model, H3, H6).
- **Internal vs. external reproducibility.** Results are internally reproducible (deterministic decode, content-hash cache, pinned revisions), but the degradation and attack conditions (C9, C10, C19–C23) use private, never-released adapters and private harmful data. Only C1–C8 (with the released C5 adapter) are externally reproducible; the headline H4/H5/H7 results must be taken from the released aggregate metrics.
- **Scope.** Findings hold on this model, these suites, this SFT recipe, this stress source, and these DPO sources and dose. They do not establish generalization off `Mistral-7B`, nor that guardrails would contain an adversary who also targets the screen.

7 Conclusion

SafeStack separates weight-level safety from guardrail safety and tests how durable the weight-level part is. Weight-level SFT alignment does nearly all of the achievable ASR reduction, at no over-refusal cost the weights themselves incur; external guardrails add little separable safety on top of it, yet become the decisive containment layer once continued fine-tuning strips the weights—which ∼411 examples do, past the base rate. Exploratory results further indicate that the training *configuration* used to strip alignment—the objective together with its learning rate and KL anchoring—matters more than the harmful data: plain SFT-on-`chosen` removes safety to the ceiling while KL-anchored DPO on the same data does not, on-family and off-family. This read carries the disclosed learning-rate confound and rests on two data sources, so the clean objective-only claim awaits the β/learning-rate arm. The practical lesson is that defense-in-depth is not redundancy for its own sake. A layer's value shows up only where the others fall short, and the one place every layer falls short together is the benign-surface (dual-use) boundary.

A Reproducibility

Provenance. Base `Mistral-7B-Instruct-v0.3` at revision `c170c708c41dac9275d15a8fff4eca08d52bab71`; aligned SFT (C5) adapter at revision `05266a9b`; stressed/attacked adapters are private, immutable-revision-pinned, and never released. Safety judge `Llama-Guard-3-1B` at `acf7aafa` (prompt version v2); guardrail `Granite-Guardian-3.1-2b`; helpfulness by a Mistral-7B rubric judge; refusal by a heuristic detector.

Determinism. Greedy decode, seed 0, `max_new_tokens` 256, bf16, no quantization at inference; bootstrap $B = 10000$, seed 0, percentile $[2.5, 97.5]$. A content-hash cache makes every result a pure function of (model, prompt, params); guardrail configuration is excluded from the generation hash so a guarded condition reuses its policy's generations. Data integrity was clean on every condition/suite (no unparsed or missing rows).

Code and data availability. Code, configuration, aggregate metrics, and sanitized/hash-only dataset previews are public at `https://github.com/kambleakash0/safestack-study` (code under Apache-2.0, written deliverables under CC-BY-4.0), with the environment pinned by a committed lockfile (Python 3.12–3.13) and per-suite manifest hashes recorded in the repository. Consistent with the responsible-use policy, raw harmful prompts/completions and the robustness-stressed and unalignment-attacked adapters are not released; the released aggregate metrics reproduce every number in this report.

Open / deferred work. A GRPO (Group Relative Policy Optimization [5]) unalignment arm and alignment-direction rungs; inference/serving benchmarks; human judge calibration; and the β/learning-rate de-confound arm, a decode-truncation check, and a behavioural-overlap audit. None gates the results above, and all are bound by the project's responsible-use policy.

References

[1] Jiang, A. et al. (2023). *Mistral 7B*. arXiv:2310.06825.

[2] Hu, E. J. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685. (ICLR 2022.)

[3] Dettmers, T. et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv:2305.14314. (NeurIPS 2023.)

[4] Rafailov, R. et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. arXiv:2305.18290. (NeurIPS 2023.)

[5] Shao, Z. et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. arXiv:2402.03300. (Introduces GRPO.)

[6] Inan, H. et al. (2023). *Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations*. arXiv:2312.06674.

[7] Grattafiori, A. et al. (2024). *The Llama 3 Herd of Models*. arXiv:2407.21783. (Includes Llama Guard 3.)

[8] Zou, A. et al. (2023). *Universal and Transferable Adversarial Attacks on Aligned Language Models*. arXiv:2307.15043. (AdvBench.)

[9] Mazeika, M. et al. (2024). *HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal*. arXiv:2402.04249. (ICML 2024.)

[10] Röttger, P. et al. (2023). *XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models*. arXiv:2308.01263. (NAACL 2024.)

[11] Taori, R. et al. (2023). *Stanford Alpaca: An Instruction-following LLaMA Model*. Stanford CRFM. `https://github.com/tatsu-lab/stanford_alpaca`.

[12] Qi, X. et al. (2023). *Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To!* arXiv:2310.03693. (ICLR 2024.)

[13] Yang, X. et al. (2023). *Shadow Alignment: The Ease of Subverting Safely-Aligned Language Models*. arXiv:2310.02949.

[14] Lermen, S., Rogers-Smith, C., Ladish, J. (2023). *LoRA Fine-tuning Efficiently Undoes Safety Training in Llama 2-Chat 70B*. arXiv:2310.20624.

[15] Qi, X. et al. (2024). *Safety Alignment Should Be Made More Than Just a Few Tokens Deep*. arXiv:2406.05946. (ICLR 2025.)

[16] Xie, T. et al. (2024). *SORRY-Bench: Systematically Evaluating Large Language Model Safety Refusal Behaviors*. arXiv:2406.14598. (ICLR 2025.)

[17] Padhi, I. et al. (2024). *Granite Guardian*. arXiv:2412.07724.

[18] Sheshadri, A. et al. (2024). *Latent Adversarial Training Improves Robustness to Persistent Harmful Behaviors in LLMs*. arXiv:2407.15549. Dataset: `LLM-LAT/harmful-dataset` (Hugging Face).

[19] Durbin, J. (2024). *toxic-dpo-v0.2*. Hugging Face dataset `unalignment/toxic-dpo-v0.2` (CC-BY-4.0).

[20] Hendrycks, D. et al. (2020). *Measuring Massive Multitask Language Understanding*. arXiv:2009.03300. (ICLR 2021.)

[21] Cobbe, K. et al. (2021). *Training Verifiers to Solve Math Word Problems*. arXiv:2110.14168.

[22] Zhou, J. et al. (2023). *Instruction-Following Evaluation for Large Language Models*. arXiv:2311.07911.

[23] Gao, L. et al. (2024). *The Language Model Evaluation Harness*. Zenodo, v0.4.3. DOI: 10.5281/zenodo.12608602.

[24] Kwon, W. et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention*. arXiv:2309.06180. (SOSP 2023.)